



Jyllands-Posten

Eksamensprojekt

2. Semester

Dataanalyse

Udarbejdet af:

**Lasse Werigin Nørgaard Abrahamsen
Christian Bressendorff
Philip Sørensen**

**Antal Tegn:
68.630 tegn**

**Antal sider:
37**

Indholdsfortegnelse

1 Problembeskrivelse	3
1.1 Afgrænsning	3
1.2 Datakilder	3
2 Datajura	3
3 GDPR og etik	3
3.1 Fælles dataansvar	4
3.2 AI etik og lovgivning	4
3.3 Cybersikkerhed	4
3.4 Databehandlere og tredjepartsteknologier	5
4 Machine learning modellen	5
4.1 Kundeadfærdsanalyse	6
4.2 Evaluering af kode	7
5 Datavisualisering	7
5.1 Evaluering af forretningsmæssige fordele	8
6 Kode	8
6.1 Datarensning	8
6.2 Klyngeanalyse	13
6.3 Churn model	18
6.3.1 Resultater og indsigt fra modellen	28
6.3.2 Model 1 Churn	28
6.3.3 Model 2: Early churn	29
6.3.4 Best, average og worst case scenarier	29
6.4 Neural network churn model	29
7 Fejlkilder og fortolkning	30
8 Konklusion	31
9 Bilag 1 - Klyngeanalyse illustrationer	31
10 Bilag 2 - Power BI Præsentation	37
11 Kildeliste	37

1 Problembeskrivelse

Jyllands-Posten anvender i stigende grad kampagnebaserede abonnementer til at tiltrække nye digitale kunder. Selvom disse kampagner genererer en høj tilgang af abonnenter, oplever virksomheden en markant churn blandt kampagneabonnenter kort tid efter kampagneperiodens udløb.

Den høje churn-rate udfordrer abonnementsforretningens langsigtede bæredygtighed og vanskeliggør en effektiv prioritering af fastholdelses- og genkøbsindsatser. Jyllands-Posten råder over omfattende data om abonnenternes adfærd, historik og engagement i kampagneperioden, men disse data anvendes i begrænset omfang til systematisk og forklarlig forudsigelse af churn.

Der er derfor behov for at undersøge, i hvilken grad Machine Learning-modeller kan anvendes til at forudsige churn blandt kampagneabonnenter baseret på deres adfærd og historik, samt hvordan disse prædiktionsmodeller kan gøres forståelige og handlingsorienterede for forretningen.

For at understøtte Jyllands-Posten anvendelse skal modelresultater og centrale churn-drivere visualiseres i en intuitiv dashboard-løsning, der kan bruges af ikke-tekniske interessenter til at identificere risikokunder, vigtige mønstre og relevante indsatsområder. Både i relation til fastholdelse og planlægning af fremtidige kampagner.

1.1 Afgrænsning

I projektet fokuseres der udelukkende på abonnenternes adfærd og historik i selve kampagneperioden. Analysen baseres på data, der er tilgængelige fra købstidspunktet og frem til kampagneperiodens udløb.

Projektet afgrænses fra at analysere kundernes adfærd efter overgang til et ordinært abonnement samt fra at inddrage længerevarende churn-forløb efter kampagneperiodens afslutning.

1.2 Datakilder

Projektet baseres på de udleveret datafiler fra Jyllands Posten, som indeholder tre datasæt:

behavior.csv: kundeadfærd i kampagneperioden

Cancellation.csv: churn-label + begrundelser

Subscription_v2.csv: abonnements- og kampagneinfo

2 Datajura

Når Jyllands-Posten arbejder med abonnentdata og digitale brugerdata, er datajura en vigtig del af projektet. Det at sikre data bliver brugt lovligt og ansvarligt. De nye oplysninger fra JP/Politikens Hus viser, at data kan indgå i flere sammenhænge, fx abonnement, brugeradfærd, annoncering, kampagnestyling og målretning. Derfor er det vigtigt at have styr på behandlingsgrundlag, samtykke, fælles dataansvar og sikkerhed JP/Politikens Hus (2025).

3 GDPR og etik

GDPR er en af de vigtigste ting at have styr på, når man arbejder med persondata. I denne case kan der være tale om abonnementsdata, adfærdsdata, kampagnedata og cookie-baserede ID'er. Jyllands-Posten skal derfor have et lovligt grundlag for at behandle data. Det kan fx være samtykke eller legitim interesse jf. artikel 6 i GDPR (JP/Politikens Hus, 2025).

I JP/Politikens Hus' retningslinjer står der, at de bruger en Consent Management Platform, også kaldet CMP, til at indhente samtykke fra brugerne. Det betyder, at brugeren skal kunne tage stilling til, hvilke former for databehandling de accepterer. Samtykke er især vigtigt, hvis data bruges til profilering, målretning eller

personaliserede annoncer. Hvis der bruges legitim interesse, skal det kunne begrundes, og der skal laves en vurdering af virksomhedens interesse over for brugerens rettigheder JP/Politikens Hus (2025).

Derudover gælder princippet om dataminimering jf. artikel 5 i GDPR. Det betyder, at Jyllands-Posten kun må bruge de data, der er nødvendige for formålet. Hvis formålet er at analysere churn, skal man derfor overveje, hvilke variable der faktisk er relevante. Man må ikke bare tage alle data med, fordi de findes. Samtidig gælder formålsbegrænsning jf. artikel 5, som betyder, at data kun må bruges til det formål, de er indsamlet til. Hvis data fx er indsamlet til abonnementshåndtering, kan de ikke uden videre bruges til avanceret profilering uden et gyldigt grundlag JP/Politikens Hus (2025).

Kunderne har også rettigheder efter GDPR jf. artikel 13-22. De har fx ret til indsigt, sletning, berigtigelse og i nogle tilfælde dataportabilitet. Det betyder, at Jyllands-Posten skal have systemer og processer, der kan håndtere, hvis en kunde ønsker at se eller få slettet sine data JP/Politikens Hus (2025).

3.1 Fælles dataansvar

En vigtig ny oplysning er, at JP/Politikens Hus i nogle situationer arbejder med fælles dataansvar jf. artikel 26 i GDPR. Det betyder, at JPPOL og en annoncør eller et mediebureau sammen kan være ansvarlige for bestemte dele af databehandlingen. Det sker fx, når en annoncør køber annoncering på JPPOL-sites og bruger egne eller tredjeparts teknologier til at indsamle data om brugere JP/Politikens Hus (2025).

Her er ansvarsfordelingen vigtig. JPPOL er ansvarlig for, at der findes et gyldigt behandlingsgrundlag, og at brugerne får information om behandlingen. Indkøberen eller annoncøren er ansvarlig for den videre behandling af data, efter at data er blevet sendt videre. Det betyder, at begge parter har ansvar, men ikke nødvendigvis for det samme JP/Politikens Hus (2025).

Hvis en bruger henvender sig om sine rettigheder, skal parterne kunne sende henvendelsen videre til den rigtige part hurtigst muligt og senest inden for 72 timer. Det viser, at dataansvar ikke kun handler om regler, men også om klare processer i praksis JP/Politikens Hus (2025).

3.2 AI etik og lovgivning

Når Jyllands-Posten bruger AI og Machine Learning til at forudsige churn, opstår der også etiske overvejelser. En churn-model kan hjælpe virksomheden med at finde kunder, der er i risiko for at opsigte deres abonnement. Det kan være positivt, fordi virksomheden kan give mere relevante tilbud og bedre kommunikation. Men det kan også blive problematisk, hvis modellen bruges til at presse eller manipulere kunder.

En vigtig risiko er bias. Hvis modellen trænes på historiske data, kan den komme til at videreføre gamle skævheder. Hvis unge kunder fx tidligere har haft høj churn, kan modellen komme til at vurdere unge som mere risikable, selvom årsagen måske ikke er fair. Derfor er det vigtigt at teste modellen og kunne forklare de vigtigste faktorer bag en churn-vurdering.

Transparens er også vigtig. Hvis kunder bliver behandlet forskelligt på baggrund af en model, bør virksomheden kunne forklare hvorfor. Det betyder ikke, at kunden skal se hele modellen, men der skal være en forståelig forklaring på, hvorfor modellen vurderer en kunde som høj risiko.

I forhold til AI-lovgivning, fx EU AI Act, vil en churn-model typisk ikke være højrisiko. Men Jyllands-Posten bør stadig dokumentere, hvordan modellen er udviklet, hvilke data der er brugt, hvordan den er evalueret, og hvad formålet er. Hvis modellen senere bruges til automatiske beslutninger, fx ændring af priser eller vilkår, kan risikoen blive højere, og så vil kravene også blive større.

3.3 Cybersikkerhed

Cybersikkerhed er også en central del af datajuraen. Når Jyllands-Posten arbejder med mange abonnent- og brugerdata, skal de beskytte data mod datalæk og uautoriseret adgang. I retningslinjerne fremgår det, at der skal bruges passende tekniske og organisatoriske foranstaltninger jf. artikel 32 i GDPR. Det gør de ved pseudonymisering, adgangsstyring og løbende test af sikkerheden JP/Politikens Hus (2025).

Det handler også om at sikre fortrolighed. Data skal være beskyttet, de må ikke ændres forkert, og systemerne skal kunne fungere stabilt. Hvis der sker et databrud, skal den part, hvis sikkerhed er berørt, anmelde bruddet til Datatilsynet jf. artikel 33, og i nogle tilfælde informere brugerne jf. artikel 34 JP/Politikens Hus (2025).

3.4 Databehandlere og tredjepartsteknologier

De nye oplysninger viser også, at JPPOL og samarbejdspartnere kan bruge databehandlere og tredjepartsteknologier. Det kan fx være annonceplatforme, analyseværktøjer eller andre teknologipartnere. Her er det vigtigt, at der findes gyldige databehandleraftaler jf. artikel 28 i GDPR, og at leverandørerne kan leve op til reglerne (JP/Politikens Hus, 2025).

Derudover fremgår det, at JPPOL-data ikke må kopieres til egne systemer eller bruges på andre sites end JPPOL-sites. Det er vigtigt, fordi det beskytter JPPOL's førstepartsdata og mindsker risikoen for, at data bliver brugt uden for det oprindelige formål JP/Politikens Hus (2025).

4 Machine learning modellen

I denne del af projektet har vi udviklet to Machine Learning-modeller, som tager udgangspunkt i Jyllands-Postens forretningsproblem omkring churn. Den første model har til formål at forudsige, hvilke kunder der cherner direkte efter kampagneperioden, og dermed også hvilke kunder der fortsætter. Den anden model bygger videre på den første og fokuserer kun på de kunder, der faktisk fortsatte efter kampagnen, for derefter at forudsige hvilke af dem der alligevel cherner efter kort tid. På den måde arbejder vi i to trin, som passer direkte til problemformuleringen i casen.

Vi starter med at indlæse det rensede datasæt `model_data.rds`, som allerede er klargjort gennem data cleaning og feature engineering. Derefter laver vi et nyt datasæt til model 1, hvor vi bevidst fjerner variable, som kan give leakage. Det gælder blandt andet `subscription_cancel_date`, `type`, `reason`, `expiration_date`, `first_campaign_day`, `last_campaign_day` og `continued_subscription`. Grunden til, at de bliver fjernet, er, at de enten direkte afslører udfaldet eller ligger for tæt op ad target-variablen. Hvis vi beholdt dem, ville modellen få information, den i praksis ikke burde have adgang til på prediction-tidspunktet, og så ville performance blive kunstigt høj. Vi beholder heller ikke `pseudo_id`, da det kun er en teknisk identifikator og ikke en reel forklarende variabel.

Selve target-variablen i model 1 er churn, som bliver lavet til en faktor med niveauerne 0 og 1. Herefter splitter vi datasættet i træningsdata og testdata med en 80/20 fordeling, hvor vi bruger stratificering på churn. Det gør vi for at sikre, at fordelingen mellem churn og ikke-churn ligner hinanden i både train og test. Efter det laver vi 5-fold cross validation på træningsdata, igen med stratificering. Formålet med det er at få en mere stabil og troværdig vurdering af modellernes performance under tuning.

I recipe-delen for model 1 forbereder vi data systematisk. Først fjerner vi `early_churn`, fordi den variabel er direkte relateret til fremtidig adfærd og derfor vil skabe leakage i churn-modellen. Derefter bruger vi `step_novel`, så modellen kan håndtere nye kategorier i testdata. Vi laver dummy-encoding af alle kategoriske variable med one-hot encoding, så modellerne kan arbejde numerisk med dem. Herefter fjerner vi `zero_variance` variable, altså variable uden reel variation, og normaliserer de numeriske variable. Til sidst bruger vi `step_downsample(churn)`, fordi churn-klasserne sandsynligvis er ubalancerede. Downsampling gør, at majoritetsklassen reduceres i træningsdata, så modellerne ikke bare lærer at gætte den største klasse hele tiden.

Vi tester fem forskellige klassifikationsmodeller. Den første er et decision tree, hvor vi tuner `tree_depth`, `min_n` og `cost_complexity`. Den anden er logistisk regression med regularisering via `glmnet`, hvor vi tuner `penalty` og `mixture`. Den tredje er random forest med `ranger`, hvor vi tuner `mtry` og `min_n`, mens antal træer er sat til 500. Den fjerde er en SVM med radial basis function, hvor vi tuner `cost` og `rbf_sigma`. Den femte er XGBoost, hvor vi tuner blandt andet antal træer, trædybde, `learning_rate` og `mtry`. På den måde sammenligner vi både simple, fortolkelige og mere avancerede modeller.

Alle modeller bliver samlet i et `workflow_set`, så de kan køres igennem den samme preprocessing pipeline. Derefter tuner vi dem med `tune_grid` på vores cross validation folds. Vi bruger flere evalueringsmål: `roc_auc`, `accuracy`, `f_meas`, `sens` og `spec`. Det er vigtigt, fordi `accuracy` alene ofte er misvisende ved ubalancerede klasser.

Vi vælger den bedste model ud fra `f_meas`, da den balancerer precision og recall og derfor er særlig relevant i en churn-case, hvor det er vigtigt både at finde churn-kunder og undgå for mange falske alarmer.

Når den bedste model er fundet, henter vi de bedste hyperparametre og finaliserer workflowet. Derefter laver vi `last_fit` på det oprindelige split, så modellen bliver trænet på hele træningsdatasættet og evalueret på testdatasættet. Til sidst indsamler vi metrics, laver en confusion matrix og en ROC-curve. Confusion matrixen giver os et konkret billede af, hvor mange kunder modellen klassificerer korrekt og forkert, mens ROC-kurven viser modellens evne til at skelne mellem churn og ikke-churn på tværs af thresholds.

Model 2 er bygget op på næsten samme måde, men her fokuserer vi kun på kunder, som allerede har continued efter kampagnen, altså hvor `churn == 0`. Formålet med denne model er at identificere early churn, altså kunder som først fortsætter, men alligevel falder fra kort tid efter. Det gør modellen mere forretningsmæssigt relevant, fordi Jyllands-Posten så ikke kun kan forstå det første frafald, men også den gruppe af kunder, der ser ud til at være vundet, men som stadig er ustabile.

I model 2 laver vi derfor et nyt datasæt `early_data`, hvor target-variablen er `early_churn`. Vi splitter igen i train og test med stratificering og laver 5-fold cross validation. I recipe'en fjerner vi churn, fordi det er parent target og ikke skal bruges som predictor i early churn-modellen. Ellers følger preprocessing stort set samme logik som i model 1: håndtering af nye kategorier, dummy-encoding, fjernelse af zero variance og normalisering. Her har vi valgt ikke at bruge downsampling i den nuværende version, men det er kommenteret i koden, så det kan tilføjes senere, hvis klasseubalancen viser sig at være et større problem.

Vi genbruger de samme fem modeltyper, så vi kan sammenligne resultaterne på tværs af de to opgaver. Også her vælger vi den bedste model på baggrund af `f_meas`, finaliserer workflowet og evaluerer på testdatasættet. Resultatet bliver igen vurderet med metrics og confusion matrix.

Samlet set er styrken ved denne ML-opsætning, at den er systematisk, sammenlignende og forholdsvis robust. Vi arbejder ikke kun med én model, men tester flere forskellige algoritmer. Vi håndterer leakage bevidst, laver preprocessing gennem recipes, bruger cross validation og hyperparameter tuning, og evaluerer modellerne med flere relevante metrics. Samtidig er modellen bygget direkte op omkring Jyllands-Postens forretningsproblem, fordi vi både forudsiger det første churn efter kampagnen og det efterfølgende early churn blandt de kunder, der i første omgang bliver fastholdt. Det gør løsningen mere brugbar i praksis, fordi resultaterne både kan bruges til at identificere risikokunder og til at planlægge mere målrettede retention-indsatser.

4.1 Kundeadfærdsanalyse

I dette afsnit kigger vi på kundeadfærd ud fra vores data i `behavior.csv`. Formålet er at forstå, hvordan brugerne opfører sig på siden, og om vi kan finde nogle mønstre, som kan bruges videre i forhold til churn.

Først laver vi data om til kundeniveau, så hver kunde kun har én række. Her laver vi nogle variable, som beskriver adfærden, fx hvor mange besøg en bruger har (`n_visits`), hvor langt de scroller (`avg_scroll`), og om de bruger mobil eller computer. Vi ser også på, hvor meget de er inde på paywall indhold, hvor de kommer fra (`search` eller `intern`), og hvor mange forskellige sider de besøger. Det giver et ret godt billede af, hvor aktive og engagerede kunderne er.

Derefter standardiserer vi data og laver en PCA. Det gør vi for at gøre data mere simpelt og samle informationen i færre variable, som stadig forklarer det meste. På den måde bliver det nemmere at arbejde videre med.

Efter det laver vi klyngeanalyse, hvor vi prøver at opdele kunderne i 3 grupper baseret på deres adfærd. Vi bruger både hierarkisk clustering og k-means for at være mere sikre på vores opdeling. Når vi har klyngerne, kigger vi på gennemsnit for hver gruppe, så vi kan forstå forskellene og lave en profil af hver kundetype.

Til sidst tester vi, om der faktisk er forskelle mellem klyngerne med ANOVA, og vi laver også nogle plots for at visualisere det.

Samlet set hjælper analysen os med at forstå kundeadfærd bedre og gør det muligt at opdele kunderne i forskellige typer, som vi senere kan bruge i vores churn analyse.

4.2 Evaluering af kode

I denne del af projektet evaluerer vi vores Machine Learning-modeller for at vurdere, hvor godt de faktisk kan forudsige churn. Vi bruger flere forskellige metrics som accuracy, ROC-AUC, sensitivity og F1-score. Grunden til, at vi ikke kun bruger én metric, er, at datasættet er skævt fordelt, hvor der typisk er flere kunder, der ikke cherner, end kunder der cherner. Derfor kan accuracy alene give et misvisende billede af modellens performance.

Vi har især fokus på F1-score, fordi den balancerer mellem precision og recall. I en forretningskontekst er det vigtigt både at identificere de kunder, der faktisk cherner (undgå false negatives), og samtidig undgå at ramme for mange kunder forkert med retention-indsatser (false positives). Hvis modellen fx overser churn-kunder, mister virksomheden potentielle indtægter, og hvis den rammer forkert, kan det føre til unødvendige marketingomkostninger.

Derudover bruger vi ROC-AUC til at vurdere modellens evne til at skelne mellem churn og ikke-churn på tværs af forskellige thresholds. Sensitivity er også relevant, da den viser, hvor stor en andel af churn-kunderne modellen faktisk fanger.

Resultaterne viser, at ensemble-modeller som random forest og XGBoost generelt performer bedst. Det giver god mening, da de er bedre til at håndtere komplekse sammenhænge og mange variable i data. Samlet set giver evalueringen et solidt grundlag for at vælge den mest brugbare model i forhold til Jyllands-Postens forretningsbehov.

5 Datavisualisering

Dashboardet er udviklet med det formål at give ledelsen og abonnementssælgerne et datadrevet overblik over churn og kundeadfærd. Visualiseringen kombinerer flere forskellige diagramtyper i Power BI, som tilsammen giver både overblik, segmentering og mulighed for at arbejde mere proaktivt med kunderne.

Centralt i dashboardet anvendes KPI cards, som viser nøgleindikatorer såsom den samlede churn rate samt andre centrale målepunkter. Disse giver brugeren et hurtigt overblik over performance og fungerer som et "high-level" beslutningsgrundlag.

Til at visualisere fordelingen af churn anvendes både bar charts og pie charts. Bar charts bruges til at sammenligne churn på tværs af forskellige segmenter, såsom målgrupper, køn og aldersgrupper, hvilket gør det nemt at identificere forskelle mellem segmenter. Pie charts anvendes til at vise den procentvise fordeling mellem kunder, der cherner, og kunder, der fortsætter, hvilket giver et hurtigt visuelt overblik over datasættets sammensætning.

Derudover anvendes et måler chart (gauge chart), som illustrerer churn rate i forhold til en ønsket målsætning. Dette gør det muligt for ledelsen hurtigt at vurdere, om virksomheden performer over eller under det ønskede niveau.

Dashboardet indeholder også et bottleneck chart, som bruges til at identificere, hvor i kunderejsen eller i segmenteringen der sker de største tab af kunder. Dette giver indsigt i, hvor indsatsen bør fokuseres for at reducere churn.

En central del af visualiseringen er klyngeanalysen, hvor kunderne er opdelt i datadrevne målgrupper (cluster_labels). Disse segmenter visualiseres og kan analyseres direkte i dashboardet, hvilket gør det muligt at identificere hvilke kundetyper der har høj churn, og hvilke der er mere loyale.

Derudover er der integreret en machine learning model, som estimerer sandsynligheden for, at en kunde vil cherne. Resultaterne fra modellen visualiseres i dashboardet og giver en prædiktiv dimension, hvor virksomheden kan identificere risikokunder, før de faktisk opsiger deres abonnement.

Dashboardet er opbygget interaktivt med mulighed for at filtrere og klikke på tværs af segmenter såsom målgrupper, køn og alder. Dette gør det muligt at dykke ned i specifikke kundesegmenter og opnå en dybere forståelse af churn-mønstre.

Samlet set kombinerer dashboardet flere visualiseringstyper – herunder KPI cards, bar charts, pie charts, gauge chart og bottleneck chart – samt avancerede analyser som klyngeanalyse og machine learning. Dette skaber et stærkt beslutningsværktøj, der gør det muligt for både ledelsen og abonnementssælgerne at arbejde mere målrettet og datadrevet med at reducere churn.

5.1 Evaluering af forretningsmæssige fordele

I en forretningsmæssig sammenhæng kan denne analyse direkte være med til at adressere Jyllands Postens problem med høj churn efter kampagneperioder. Ved at bruge Machine Learning til at forudsige churn, får virksomheden mulighed for at identificere hvilke kunder, der er i risiko for at opsige deres abonnement allerede inden det sker. Det gør det muligt at arbejde proaktivt i stedet for reaktivt.

Samtidig giver analysen indsigt i, hvilke faktorer der driver churn. Ved at kombinere abonnementsdata og adfærdsdata kan Jyllands Posten forstå, hvilke typer kunder der cherner, og hvad der kendetegner dem. Fx kan det være lav aktivitet, få besøg eller manglende engagement med indhold. Denne viden gør det muligt at målrette indsatsen, så man ikke bruger ressourcer på alle kunder, men fokuserer på dem med størst risiko.

Kundesegmenteringen spiller også en vigtig rolle, da den opdeler kunderne i forskellige grupper baseret på deres adfærd. Det gør det muligt at tilpasse både kommunikation, tilbud og onboarding. Fx kan nye kunder med lav aktivitet få ekstra hjælp til at komme i gang, mens mere aktive kunder kan få mere relevant indhold eller premium tilbud. På den måde kan Jyllands-Posten skabe mere værdi for den enkelte kunde og dermed øge sandsynligheden for fastholdelse.

Derudover gør visualiseringerne i Power BI det muligt at omsætte analysen til et konkret værktøj for forretningen. I stedet for kun at have tekniske modelresultater, får Jyllands Posten et dashboard, hvor de kan følge churn, identificere risikokunder og se mønstre i data. Et gør løsningen anvendelig for ikke-tekniske brugere og understøtter bedre beslutningstagning. Ud fra det kan det konkluderes, at Jyllands Posten kan arbejde mere datadrevet med både fastholdelse og fremtidige kampagner. I stedet for at fokusere på volumen af nye abonnenter, kan de i højere grad fokusere på kvaliteten af kunderne og deres langsigtede værdi. Dette kan være med til at reducere churn, øge kundeloyaliteten og skabe en mere bæredygtig abonnementsforretning.

6 Kode

6.1 Datarensning

```
pacman::p_load(tidyverse, lubridate, forcats, readr)

# 1. Indlæs data
cancellation <- read.csv("data/cancellation.csv")
subscription <- read.csv("data/subscription_v2.csv", sep = ";")

# Sæt import_date baseret på nyeste data i filen
import_date <- as.Date("2026-03-24")

# 2. Behold seneste observation pr. bruger
cancellation <- cancellation %>%
  mutate(expiration_date = ymd(expiration_date)) %>%
  group_by(pseudo_id) %>%
  slice_max(expiration_date, n = 1, with_ties = FALSE) %>%
  ungroup()

subscription <- subscription %>%
  mutate(subscription_cancel_date = dmy(subscription_cancel_date)) %>%
  group_by(pseudo_id) %>%
  slice_max(subscription_cancel_date, n = 1, with_ties = FALSE) %>%
  ungroup()

# 3. Merge + churn logik
merged_data <- subscription %>%
```



```

left_join(cancellation, by = "pseudo_id") %>%
mutate(
  subscription_cancel_date = as.Date(subscription_cancel_date),
  first_campaign_day = dmy(first_campaign_day),
  last_campaign_day = dmy(last_campaign_day),
  expiration_date = as.Date(expiration_date)
) %>%
mutate(
  churn = case_when(
    is.na(expiration_date) ~ 0,
    expiration_date <= last_campaign_day ~ 1,
    expiration_date > last_campaign_day ~ 0
  )
) %>%
mutate(
  continued_subscription = 1 - churn
) %>%
mutate(
  early_churn = case_when(
    continued_subscription == 1 &
      !is.na(expiration_date) &
      expiration_date <= last_campaign_day + 90 ~ 1,
    continued_subscription == 1 ~ 0,
    continued_subscription == 0 ~ 0
  )
)

# 4. Feature engineering
model_data <- merged_data %>%
mutate(
  # Parse datoer
  order_date = as.Date(ymd_hms(order_date)),
  birthdate = as.Date(parse_date_time(birthdate, orders = c("dmy", "ymd", "mdy"))),
  usr_created = dmy(usr_created),

  # Afledte variable
  age = as.numeric(import_date - birthdate) / 365,
  kundetid_dage = as.numeric(order_date - usr_created)
) %>%

# Fjern ugyldige observationer
filter(
  !is.na(birthdate),
  !is.na(usr_created),
  !is.na(order_date),
  previous_trials <= 100,
  age >= 15,
  age <= 105,
  kundetid_dage >= 0
) %>%

# Rens kategoriske variable
mutate(
  churn = factor(churn),
  type = fct_na_value_to_level(factor(type), "Ingen afmelding"),

```

```

    reason = fct_na_value_to_level(factor(reason), "Ingen afmelding"),
    koen = fct_recode(factor(koen), "Ikke oplyst" = ""),
    permission_given_order = factor(permission_given_order),
    permission_given_today = factor(permission_given_today),

    permission_given_order = factor(if_else(permission_given_order == "true", 1, 0)),
    permission_given_today = factor(if_else(permission_given_today == "true", 1, 0)),

    expiration_date = replace_na(expiration_date, as.Date("3000-01-01"))
  ) %>%

# Kundelevetid grupper
mutate(
  kundetid_gruppe = case_when(
    kundetid_dage <= 7 ~ "0-7 dage",
    kundetid_dage <= 30 ~ "8-30 dage",
    kundetid_dage <= 180 ~ "1-6 måneder",
    TRUE ~ "6+ måneder"
  ) %>% factor()
) %>%

# Fjern hjælpe-variable og rå datoer
select(
  -kundetid_dage,
  -order_date,
  -birthdate,
  -usr_created,
)

# 5. Tjek resultatet
glimpse(model_data)

```

```

Rows: 1,259
Columns: 22
$ pseudo_id          <chr> "003a92354fb88e0e3e411bc290291b6a", "004b2a16~
$ account_active_days <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ subscription_cancel_date <date> 2025-05-11, 2025-04-29, 2025-03-18, 2025-05-~
$ koen                <fct> Kvinde, Kvinde, Mand, Kvinde, Kvinde, Mand, M~
$ order_trackertag     <chr> "utm_campaign=Marts25&utm_content=b&utm_mediu~
$ permission_given_order <fct> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ permission_given_today <fct> 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, ~
$ previous_subscriptions <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns    <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1,~
$ previous_trials       <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0,~
$ first_campaign_day    <date> 2025-03-05, 2025-03-05, 2025-03-18, 2025-03-~
$ last_campaign_day     <date> 2025-05-04, 2025-05-04, 2025-05-17, 2025-05-~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, ~
$ newsletters_after_order <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, ~
$ type                 <fct> Andet, Andet, Andet, Produkt, Tid, Andet, Tid~
$ reason               <fct> kunden ønsker ikke at oplyse, Ingen årsag - i~
$ expiration_date       <date> 2025-06-04, 2025-05-04, 2025-05-17, 2025-06-~
$ churn                <fct> 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, ~
$ continued_subscription <dbl> 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, ~
$ early_churn           <dbl> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, ~
$ age                  <dbl> 76.25753, 69.70685, 39.67397, 45.61096, 74.21~

```

```
$ kundetid_gruppe          <fct> 6+ måneder, 6+ måneder, 0-7 dage, 0-7 dage, 6~
```

```
# Overblik over churn
```

```
model_data %>%  
  count(churn) %>%  
  mutate(prop = n / sum(n))
```

```
# A tibble: 2 x 3
```

```
  churn      n prop  
  <fct> <int> <dbl>  
1 0         731 0.581  
2 1         528 0.419
```

```
# Outlier tjek - dem har vi fjernet fra model_data
```

```
outliers <- merged_data %>%  
  mutate(  
    order_date = as.Date(ymd_hms(order_date)),  
    birthdate  = as.Date(parse_date_time(birthdate, orders = c("dmy", "ymd", "mdy"))),  
    usr_created = dmy(usr_created),  
  
    age = as.numeric(Sys.Date() - birthdate) / 365,  
    kundetid_dage = as.numeric(order_date - usr_created)  
  ) %>%  
  filter(  
    age < 15 | age > 105 |  
    kundetid_dage < 0 |  
    is.na(age) |  
    is.na(kundetid_dage)  
  ) %>%  
  select(pseudo_id, birthdate, age, usr_created, order_date, kundetid_dage)  
  
# Kig outliers  
glimpse(outliers)
```

```
Rows: 19
```

```
Columns: 6
```

```
$ pseudo_id    <chr> "04a08cd300c7b1c80803fac3de32df92", "10dccc12a352afbf406~  
$ birthdate    <date> 2026-01-01, 1992-05-02, 2025-03-12, 1940-10-10, 1883-05~  
$ age          <dbl> 0.339726, 34.030137, 1.147945, 85.624658, 143.106849, 39~  
$ usr_created  <date> 2013-04-22, NA, 2023-05-27, NA, 2025-03-08, 2025-03-17,~  
$ order_date   <date> 2025-03-12, 2025-03-04, 2025-03-21, 2025-03-15, 2025-03~  
$ kundetid_dage <dbl> 4342, NA, 664, NA, 0, -2, 3084, 3559, -32, -218, NA, -1,~
```

```
# Gem data
```

```
saveRDS(model_data, "data/model_data.rds")
```

Datarensning

Formålet med datarensningen er at skabe et konsistent, veldokumenteret og reproducerbart datasæt, som kan anvendes direkte i den efterfølgende churn model. Processen er opdelt i fire trin, som bygger logisk videre på hinanden.

Trin 1

I første trin indlæses de to datafiler cancellation.csv og subscription_v2.csv. Samtidig oprettes variablen import_date med datoen 2026 03 24, som er den dato, hvor data er eksporteret fra Jyllands Posten. Denne dato

gemmes eksplicit for at sikre reproducerbarhed og bruges senere som reference i beregninger, fx ved udregning af alder.

Trin 2

En udfordring i data er, at en bruger kan have afmeldt sig og genabonneret flere gange. Det betyder, at datasættet kan indeholde flere rækker for samme bruger ID. Da analysen fokuserer på den aktuelle churn status, filtreres der, så kun det seneste abonnement pr. bruger bevares. Dette sikrer, at hver bruger kun optræder én gang i datasættet.

Trin 3

De to datasæt samles herefter med et `left_join` på bruger ID, så alle abonnenter bevares, også selvom de aldrig har opsagt deres abonnement. Alle dato kolonner konverteres samtidig til korrekt datoformat for at sikre konsistente beregninger.

Churn defineres ved hjælp af `case_when()` ud fra følgende logik:

Hvis `expiration_date` er NA, har brugeren aldrig opsagt $\rightarrow$ churn = 0

Hvis `expiration_date` er før eller lig med kampagnens slutdato $\rightarrow$ churn = 1

Hvis `expiration_date` ligger efter kampagnens slutdato $\rightarrow$ churn = 0

Variablen `continued_subscription` defineres som det inverse af churn og anvendes til afledte analyser.

Derudover defineres `early_churn`, som kun kan være 1 for brugere, der ellers fortsætter abonnementet. Early churn defineres konkret som opsigelse inden for 90 dage efter kampagnens slutdato. Alle churnede brugere får automatisk værdien 0.

Trin 4

Variablen `order_date` indeholder både dato og klokkeslæt og parses derfor med `ymd_hms()`, hvorefter klokkeslættet fjernes, så kun datoen bevares. `birthdate` parses med en fleksibel dato parser, da formatet er konsistent i data.

På baggrund af disse datoer konstrueres to nye variable:

Alder, beregnet som antal dage fra fødselsdato til `import_date`, divideret med 365

Kundetid_dage, beregnet som antal dage fra brugeroprettelse til første ordre

Herefter foretages en række filtreringer for at sikre datakvalitet:

Rækker med manglende kritiske datoer fjernes

Observationer med mere end 100 tidligere forsøgsabonnementer ekskluderes

Alder filtreres til intervallet 15–105 år

Observationer hvor ordre er placeret før brugeroprettelse fjernes

Manglende værdier i `type` og `reason` (som kun forekommer for brugere, der aldrig har afmeldt sig) omkodes til kategorien “ingen afmelding”, så modellen kan behandle dem som en selvstændig gruppe. Tomme værdier i `køn` omkodes til “ikke oplyst”. `permission_given_*` variable konverteres til faktorer.

For abonnementer uden udløbsdato erstattes `expiration_date` med år 3000, som en teknisk markør for “ingen udløbsdato”, uden at introducere NA, som kan skabe problemer i modelleringen.

`kundetid_dage` grupperes i fire kategorier i stedet for at fastholdes som en præcis numerisk værdi. Det gør modellen mere robust over for outliers. Til sidst fjernes de rå dato variable samt `kundetid_dage`, da de nu er erstattet af afledte features.

Trin 5

Som afslutning på datarensningen undersøges fordelingen af churn med `count(churn)` og procentandele. Dette gøres for at vurdere, om datasættet er skævt fordelt, da en kraftig ubalance kan påvirke modellens læring.

De observationer, der blev fjernet under rensningen, samles i objektet outliers. Disse bruges ikke i modellen, men undersøges separat for at afgøre, om de skyldes tilfældige fejl eller systematiske mønstre i data.

Til sidst gemmes det rensede datasæt som en RDS fil, så det kan anvendes direkte i de efterfølgende modeller uden tab af datatyper eller struktur.

6.2 Klyngeanalyse

```
pacman::p_load(tidyverse, DataExplorer, ggpubr)

# Load data
behavior <- read.csv("data/behavior.csv")
model_data <- readRDS("data/model_data.rds")

# Vi formaterer data så vi kan join det med vores model_data
customer_behavior <- behavior %>%
  group_by(pseudo_id) %>%
  summarise(
    n_visits = n(),
    avg_scroll = mean(scroll_depth, na.rm = TRUE),
    share_mobile = mean(dvce_type == "Mobile"),
    share_desktop = mean(dvce_type == "Computer"),
    share_restricted = mean(page_restricted == "yes"),
    share_search = mean(refr_medium == "search"),
    share_internal = mean(refr_medium == "internal"),
    n_unique_pages = n_distinct(webpage_id)
  )

saveRDS(customer_behavior, "data/coolbehavior.rds")

# Vi vælger værdier fra model_data som skal bruges til klyngeanalysen til join
subscription_features <- model_data %>%
  select(
    pseudo_id,
    account_active_days,
    previous_subscriptions,
    previous_campaigns,
    previous_trials,
    newsletters_before_order,
    newsletters_after_order,
    churn
  )

# Vi joiner tabellerne sammen til klyngeanalysen
customer_df <- subscription_features %>%
  left_join(customer_behavior, by = "pseudo_id")

# Indsæt 0 i stedet for NA for de 70 "Spøgelses-brugere"
# Siden der er nogle brugere som ikke har behavior værdier,
# sætter vi dem til 0 for at beholde rækkerne
customer_df <- customer_df %>%
  mutate(across(
    c(n_visits, n_unique_pages, avg_scroll, starts_with("share_")),
    ~replace_na(.x, 0)
  ))
```

```

))

# Nu fjerner vi pseudo_ID
cluster_input <- customer_df %>%
  select(-pseudo_id, -churn)

# Tjek eventuelt histogram over tallene i bilag 1

# Kør scale til klyngeanalysen
customer_scaled <- scale(cluster_input)

# Principal component analyse - kig summary i bilag 1
pca <- prcomp(customer_scaled, scale = TRUE)

# Se PCA plot i bilag 1

# Hierakisk clustering - kig dendogram i bilag 1

# hc_complete <- hclust(dist(pca$x[,1:4]), method = "complete")
# plot(hc_complete)
#
# customer_df <- customer_df %>%
#   filter(complete.cases(select(., -pseudo_id, -churn)))
#
# customer_df$cluster_hc <- cutree(hc_complete, 5)
#
# table(customer_df$cluster_hc)

hc_ward <- hclust(dist(pca$x[,1:4]), method = "ward.D2")

customer_df <- customer_df %>%
  filter(complete.cases(select(., -pseudo_id, -churn)))

customer_df$cluster_hc <- cutree(hc_ward, 5)

table(customer_df$cluster_hc)

```

```

  1  2  3  4  5
265 342 407 198 47

```

```

# K-means clustering
set.seed(8)

km <- kmeans(pca$x[,1:4], centers = 5, nstart = 20)

customer_df$cluster_km <- km$cluster

table(customer_df$cluster_km)

```

```

  1  2  3  4  5
239 418 91 365 146

```

```
# compare methods
table(customer_df$cluster_km, customer_df$cluster_hc)
```

```
      1    2    3    4    5
1  59    0   27  153    0
2  48    5  361    1    3
3    7   75    9    0    0
4  49  262    9   44    1
5 102    0    1    0   43
```

```
# Klyngeprofilering
customer_df$cluster_km <- as.factor(customer_df$cluster_km)

cluster_profile <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(across(where(is.numeric), mean, na.rm = TRUE))
```

Warning: There was 1 warning in `summarise()`.

- i In argument: `across(where(is.numeric), mean, na.rm = TRUE)`.
- i In group 1: `cluster\_km = 1`.

Caused by warning:

- ! The `...` argument of `across()` is deprecated as of dplyr 1.1.0. Supply arguments directly to `.fns` through an anonymous function instead.

```
# Previously
across(a:b, mean, na.rm = TRUE)

# Now
across(a:b, \(x) mean(x, na.rm = TRUE))
```

```
# Tjek ggplot visualiseringer og ANOVA tests i bilag 1
```

```
# Churn analyse
churn_by_cluster <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(
    churn_rate = mean(churn == 1, na.rm = TRUE),
    n = n()
  )

print(churn_by_cluster)
```

```
# A tibble: 5 x 3
  cluster_km churn_rate      n
  <fct>      <dbl> <int>
1 1          0.469   239
2 2          0.419   418
3 3          0.385    91
4 4          0.362   365
5 5          0.507   146
```

```
# Print summary af klynger
cluster_summary_long <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(across(where(is.numeric), \(x) mean(x, na.rm = TRUE))) %>%
  pivot_longer(cols = -cluster_km, names_to = "variable", values_to = "mean_value") %>%
  pivot_wider(names_from = cluster_km, names_prefix = "Cluster_", values_from = mean_value)

print(cluster_summary_long, n = 50)
```

```
# A tibble: 15 x 6
```

	variable <chr>	Cluster_1 <dbl>	Cluster_2 <dbl>	Cluster_3 <dbl>	Cluster_4 <dbl>	Cluster_5 <dbl>
1	account_active_days	640.	369.	469.	280.	990.
2	previous_subscriptions	3.65	1.38	2.32	0.989	12.3
3	previous_campaigns	2.05	0.821	1.27	0.573	5.84
4	previous_trials	1.28	0.486	0.901	0.384	6.24
5	newsletters_before_order	0.331	0.292	0.330	0.395	3.80
6	newsletters_after_order	0.377	0.340	0.363	0.471	3.92
7	n_visits	617.	126.	4.02	78.2	228.
8	avg_scroll	0.299	0.423	0.0281	0.423	0.400
9	share_mobile	0.686	0.0644	0.168	0.904	0.414
10	share_desktop	0.271	0.908	0.0963	0.0674	0.544
11	share_restricted	0.277	0.348	0.0169	0.394	0.357
12	share_search	0.0490	0.0683	0.00139	0.0407	0.0353
13	share_internal	0.568	0.538	0.0545	0.423	0.507
14	n_unique_pages	614.	126.	4.02	77.7	228.
15	cluster_hc	3.15	2.78	2.02	2.14	2.19

```
# definering af 5 labels baseret på profilerne
customer_df <- customer_df %>%
  mutate(cluster_label = case_when(
    cluster_km == 1 ~ "Power-brugeren (Mest mobil)",
    cluster_km == 2 ~ "Desktop-traditionalisten",
    cluster_km == 3 ~ "Veteranen (Høj historik)",
    cluster_km == 4 ~ "Mobil-nykommeren",
    cluster_km == 5 ~ "Spørgelses-brugeren (Lav aktivitet)",
    TRUE ~ "Andet"
  ))
```

```
# Tjek den nye fordeling
table(customer_df$cluster_label, customer_df$churn)
```

	0	1
Desktop-traditionalisten	243	175
Mobil-nykommeren	233	132
Power-brugeren (Mest mobil)	127	112
Spørgelses-brugeren (Lav aktivitet)	72	74
Veteranen (Høj historik)	56	35

```
# Gem mapping til din Tidymodels model
cluster_mapping <- customer_df %>%
  select(pseudo_id, cluster_label)
```



```
saveRDS(cluster_mapping, "data/cluster_mapping.rds")

# Tjek fordelingen
table(customer_df$cluster_label)
```

Desktop-traditionalisten	Mobil-nykommeren
418	365
Power-brugeren (Mest mobil)	Spørgelses-brugeren (Lav aktivitet)
239	146
Veteranen (Høj historik)	
91	

PCA er anvendt til dimensionsreduktion ved at omdanne de 14 oprindelige variable til ukorrelerede hovedkomponenter, som er rangeret efter den andel af den samlede varians, de forklarer. Scree-plottet viser, at de første fem principalkomponenter har egenverdier over 1. Derudover ses et markant fald i kurvens hældning efter den sjette komponent. De første fem hovedkomponenter forklarer således størstedelen af den samlede varians og danner derfor grundlaget for den efterfølgende klyngeanalyse. Biplottet bekræfter, at observationerne fordeler sig som forventet langs de to første principalkomponenter, PC1 og PC2. Der ses en tydelig adskillelse mellem besøgsadfærd, enhedspræferencer og historiske variabler, hvilket understøtter, at PCA-løsningen er analytisk velfungerende.

Hierarkisk clustering

Den hierarkiske clustering er gennemført ved anvendelse af Ward.D2-metoden på de fem første principalkomponenter. Metoden er valgt, da den har fokus på at minimere variationen inden for klyngerne ved hvert sammenlægningskridt og derfor typisk resulterer i relativt kompakte og veldefinerede segmenter. Dendrogrammet er anvendt til at vurdere det naturlige antal klynger i datasættet og peger på, at en opdeling i fem klynger giver den mest meningsfulde løsning.

K-means clustering

På baggrund af den hierarkiske analyse er der efterfølgende gennemført en K-means clustering med $k=5$. For at sikre en stabil løsning er algoritmen kørt med 20 tilfældige startpunkter, hvilket reducerer risikoen for konvergens mod et lokalt minimum. En sammenligning mellem K-means løsningen og den hierarkiske clustering viser høj overensstemmelse i fire ud af fem klynger, hvilket indikerer at segmenteringen er robust på tværs af metoder.

Klyngeprofilering

For at opnå en klar forståelse af de fem identificerede segmenter er der beregnet gennemsnitsværdier for alle numeriske variabler inden for hver klynge. Disse klyngeprofiler er efterfølgende visualiseret ved hjælp af boxplots for de mest adskillende variabler, herunder besøgsfrekvens, scrollldybde, kontodage og tidligere abonnementer. Visualiseringerne giver et intuitivt overblik over både niveauet og variationen i brugeradfærd inden for hvert segment.

Boxplottene viser, at klynge 1 adskiller sig tydeligt ved en høj og relativt varieret besøgsfrekvens, hvilket indikerer et segment med intensiv og uensartet brug af platformen. Klynge 4 og 5 udviser derimod meget lav aktivitet på tværs af variablerne. Den brede spredning i klynge 3 afspejler et mere heterogent segment med stor variation i både besøgsintensitet og historik. For scrollldybde har klynge 5 den laveste gennemsnitsværdi, hvilket harmonerer med segmentets generelt lave engagement. Klynge 3 skiller sig samtidig markant ud i forhold til tidligere abonnementer, hvilket understreger den lange historik som et centralt kendetegn ved dette segment.

For yderligere at validere, om de observerede forskelle mellem segmenterne er statistisk holdbare, er der gennemført ensidede variansanalyser (ANOVA) for alle inkluderede variabler med klyngetilhørsforhold som grupperende faktor. Resultaterne viser, at alle otte variabler adskiller sig statistisk signifikant på tværs af klyngerne ($p < 0,001$). Især besøgsfrekvens, tidligere abonnementer og tidligere trials udviser markante forskelle mellem segmenterne, mens scrollldybde og antal modtagne nyhedsbreve ligeledes bidrager til adskillelsen, omend i mindre grad. Samlet set bekræfter både klyngeprofilerne og ANOVA-resultaterne, at de fem segmenter repræsenterer

systematiske og meningsfulde forskelle i brugeradfærd og dermed udgør et solidt grundlag for videre analyse og fortolkning.

Churn-analyse

For at undersøge segmenternes relation til kundefrafald er churn-raten beregnet som andelen af kunder inden for hvert segment, der har churnet i observationsperioden. På baggrund af klyngeprofilerne er de fem segmenter tildelt navngivne labels, der afspejler deres dominerende adfærdsmæssige kendetegn: Power-brugeren, Desktop-traditionalisten, Veteranen, Mobil-nykommeren og Spøgelses-brugeren.

Churn-raten varierer på tværs af segmenterne, og Veteranen udviser den højeste frafaldrate. Dette er bemærkelsesværdigt, da segmentet har den mest intensive historik med platformen målt på tidligere abonnemeter, kampagner og trials. Det kan indikere, at gentagen prøveadfærd ikke konverterer til et stabilt og fastholdt abonnement, og at segmentet er præget af kunder med lav loyalitet på trods af høj eksponering. Power-brugeren udviser ligeledes en høj churn-rate trods intenst engagement, hvilket peger på, at høj aktivitet ikke nødvendigvis er ensbetydende med fastholdelse. Mobil-nykommeren har den laveste churn-rate, hvilket kan afspejle, at nyere kunder endnu befinder sig i en tidlig fase af deres kunderejse. Klynge-labelsmappingen er gemt og videreføres som input til den efterfølgende prædiktive modellering i Tidymodels.

6.3 Churn model

```
pacman::p_load(
  tidyverse, tidymodels, themis, ranger, glmnet,
  kernlab, kknn, xgboost, naivebayes, rpart, future, vip
)

# Indlæs data
model_data <- readRDS("data/model_data.rds")
cluster_mapping <- readRDS("data/cluster_mapping.rds")
coolbehavior <- readRDS("data/coolbehavior.rds")

### dashboard data til power BI
final_dashboard_data <- model_data %>%
  left_join(cluster_mapping, by = "pseudo_id")

final_dashboard_data$page <- round(final_dashboard_data$page)

merged_dashboard_data <- final_dashboard_data %>%
  left_join(coolbehavior, by = "pseudo_id")

# Indsæt 0 i stedet for NA for de 70 "Spøgelses-brugere"
merged_dashboard_data <- merged_dashboard_data %>%
  mutate(across(
    c(n_visits, n_unique_pages, avg_scroll, starts_with("share_")),
    ~replace_na(.x, 0)
  ))

write_csv(merged_dashboard_data, "data/dashboard_data.csv")

# Join cluster labels and clean initial data
model_data_clean <- model_data %>%
  left_join(cluster_mapping, by = "pseudo_id") %>%
  mutate(cluster_label = as.factor(cluster_label)) %>%
  select(
    -pseudo_id,
```

```

    -subscription_cancel_date,
    -type,
    -reason,
    -expiration_date,
    -order_trackertag,
    -first_campaign_day,
    -last_campaign_day,
    -continued_subscription
  ) %>%
  mutate(churn = factor(churn, levels = c("1", "0")))

# Tjek data
glimpse(model_data_clean)

```

```

Rows: 1,259
Columns: 14
$ account_active_days      <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ koen                     <fct> Kvinde, Kvinde, Mand, Kvinde, Kvinde, Mand, M~
$ permission_given_order   <fct> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ permission_given_today   <fct> 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, ~
$ previous_subscriptions    <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns       <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1,~
$ previous_trials          <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0,~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ newsletters_after_order  <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ churn                    <fct> 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, ~
$ early_churn              <dbl> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, ~
$ age                      <dbl> 76.25753, 69.70685, 39.67397, 45.61096, 74.21~
$ kundetid_gruppe         <fct> 6+ måneder, 6+ måneder, 0-7 dage, 0-7 dage, 6~
$ cluster_label           <fct> Power-brugeren (Mest mobil), Spørgelses-bruger~

```

```

# Gem data
saveRDS(model_data_clean, "data/model_data_clean.rds")

# Split
set.seed(8)
split <- initial_split(model_data_clean, prop = 0.8, strata = churn)
train_data <- training(split)
test_data <- testing(split)

# CV
folds <- vfold_cv(train_data, v = 5, strata = churn)

# Recipe
rec <- recipe(churn ~ ., data = train_data) %>%
  step_rm(early_churn) %>%
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors(), one_hot = TRUE) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors()) %>%
  step_smote(churn)
# step_upsample(churn)
# step_downsample(churn)

# Models

```

```

decision_tree_spec <- decision_tree(
  tree_depth = tune(),
  min_n = tune(),
  cost_complexity = tune()
) %>%
  set_engine("rpart") %>%
  set_mode("classification")

log_reg_spec <- logistic_reg(
  penalty = tune(),
  mixture = tune()
) %>%
  set_engine("glmnet") %>%
  set_mode("classification")

rand_forest_spec <- rand_forest(
  mtry = tune(),
  min_n = tune(),
  trees = 500
) %>%
  set_engine("ranger", importance = "permutation") %>%
  set_mode("classification")

svm_rbf_spec <- svm_rbf(
  cost = tune(),
  rbf_sigma = tune()
) %>%
  set_engine("kernlab") %>%
  set_mode("classification")

xgb_spec <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  learn_rate = tune(),
  mtry = tune()
) %>%
  set_engine("xgboost") %>%
  set_mode("classification")

# Workflow
workflow_set_obj <- workflow_set(
  preproc = list(rec = rec),
  models = list(
    decision_tree = decision_tree_spec,
    logistic_reg   = log_reg_spec,
    rand_forest    = rand_forest_spec,
    svm_rbf        = svm_rbf_spec,
    xgboost        = xgb_spec
  )
)

metrics <- metric_set(roc_auc, accuracy, f_meas, sens, spec)

# Train

```

```

plan(multisession)

set.seed(8)

results <- suppressMessages(suppressWarnings(
  workflow_set_obj %>%
    workflow_map(
      "tune_grid",
      resamples = folds,
      grid = 10,
      metrics = metrics,
      verbose = FALSE,
      control = control_grid(
        save_pred = TRUE,
        save_workflow = TRUE
      )
    )
  ))

```

maximum number of iterations reached 1.360417e-05 -8.598677e-13maximum number of iterations reached 0.0

```

plan(sequential)

# Best model
best_model_id <- results %>%
  rank_results(select_best = TRUE) %>%
  filter(.metric == "f_meas") %>%
  slice_max(mean, n = 1) %>%
  pull(wflow_id)

best_tuned <- results %>%
  extract_workflow_set_result(best_model_id) %>%
  select_best(metric = "f_meas")

final_wf <- results %>%
  extract_workflow(best_model_id) %>%
  finalize_workflow(best_tuned)

# Final evaluation
final_fit <- final_wf %>%
  last_fit(split, metrics = metrics)

collect_metrics(final_fit)

```

```

# A tibble: 5 x 4
  .metric .estimator .estimate .config
  <chr>    <chr>         <dbl> <chr>
1 accuracy binary         0.771 pre0_mod0_post0
2 f_meas  binary         0.743 pre0_mod0_post0
3 sens    binary         0.792 pre0_mod0_post0
4 spec    binary         0.755 pre0_mod0_post0
5 roc_auc binary         0.849 pre0_mod0_post0

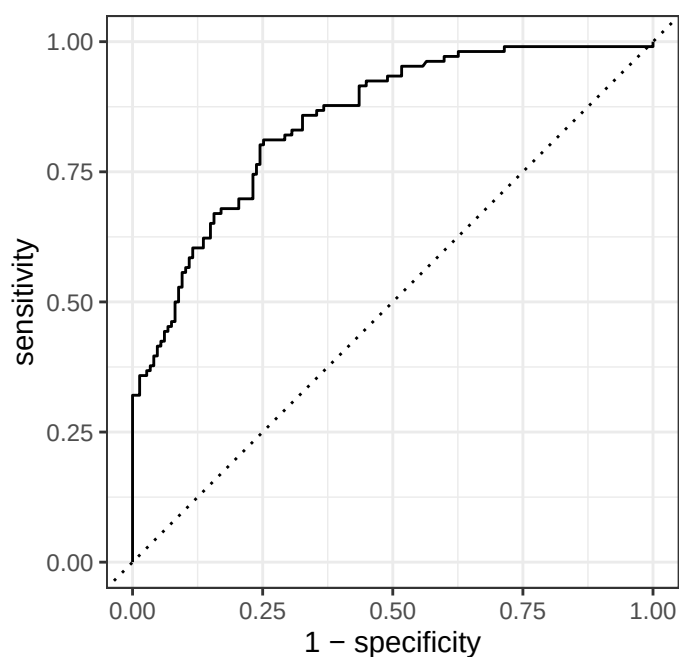
```

```
test_preds <- collect_predictions(final_fit)

test_preds %>%
  conf_mat(truth = churn, estimate = .pred_class)
```

```
      Truth
Prediction 1  0
      1  84 36
      0  22 111
```

```
test_preds %>%
  roc_curve(truth = churn, .pred_1) %>%
  autoplot()
```



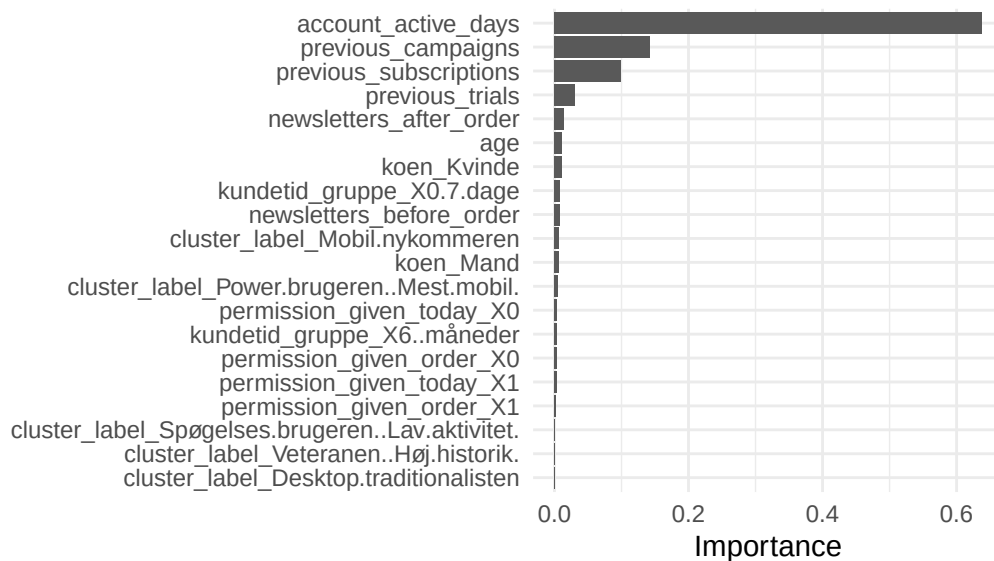
```
best_model_id
```

```
[1] "rec_xgboost"
```

```
final_fit %>%
  extract_fit_parsnip() %>%
  vip(num_features = 20) +
  theme_minimal() +
  labs(title = "Hvad driver Churn?",
       subtitle = "Kolonner rangeret efter vigtighed (Random Forest)")
```

Hvad driver Churn?

Kolonner rangeret efter vigtighed (Ra



```
# MODEL 2: EARLY CHURN

# Only customers who continued
early_data <- model_data_clean %>%
  filter(churn == "0") %>%
  mutate(early_churn = factor(early_churn, levels = c("0", "1")))

# Split
set.seed(8)
split_early <- initial_split(early_data, prop = 0.8, strata = early_churn)
train_early <- training(split_early)
test_early <- testing(split_early)

# CV
folds_early <- vfold_cv(train_early, v = 5, strata = early_churn)

# Recipe
rec_early <- recipe(early_churn ~ ., data = train_early) %>%
  step_rm(churn) %>% # remove parent target
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors(), one_hot = TRUE) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors())
# step_downsample(early_churn)

# Workflow (reuse same models!)
workflow_set_early <- workflow_set(
  preproc = list(rec = rec_early),
  models = list(
    decision_tree = decision_tree_spec,
    logistic_reg = log_reg_spec,
    rand_forest = rand_forest_spec,
    svm_rbf = svm_rbf_spec,
```

```

    xgboost      = xgb_spec
  )
)

# Train
plan(multisession)

set.seed(8)
results_early <- workflow_set_early %>%
  workflow_map(
    "tune_grid",
    resamples = folds_early,
    grid = 10,
    metrics = metrics
  )

```

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

i Creating pre-processing data to finalize 1 unknown parameter: "mtry"

maximum number of iterations reached 0.003258396 0.00318488maximum number of iterations reached 0.00024

i Creating pre-processing data to finalize 1 unknown parameter: "mtry"

```

plan(sequential)

# Best model
best_model_id_early <- results_early %>%
  rank_results(select_best = TRUE) %>%
  filter(.metric == "f_meas") %>%
  slice_max(mean, n = 1) %>%
  pull(wflow_id)

```



```

best_tuned_early <- results_early %>%
  extract_workflow_set_result(best_model_id_early) %>%
  select_best(metric = "f_meas")

final_wf_early <- results_early %>%
  extract_workflow(best_model_id_early) %>%
  finalize_workflow(best_tuned_early)

# Final evaluation
final_fit_early <- final_wf_early %>%
  last_fit(split_early, metrics = metrics)

collect_metrics(final_fit_early)

```

```

# A tibble: 5 x 4
  .metric .estimator .estimate .config
  <chr>    <chr>         <dbl> <chr>
1 accuracy binary         0.837 pre0_mod0_post0
2 f_meas  binary         0.888 pre0_mod0_post0
3 sens    binary         0.95  pre0_mod0_post0
4 spec    binary         0.596 pre0_mod0_post0
5 roc_auc binary         0.869 pre0_mod0_post0

```

```

test_preds_early <- collect_predictions(final_fit_early)

test_preds_early %>%
  conf_mat(truth = early_churn, estimate = .pred_class)

```

```

      Truth
Prediction 0 1
      0 95 19
      1  5 28

```

```
best_model_id_early
```

```
[1] "rec_rand_forest"
```

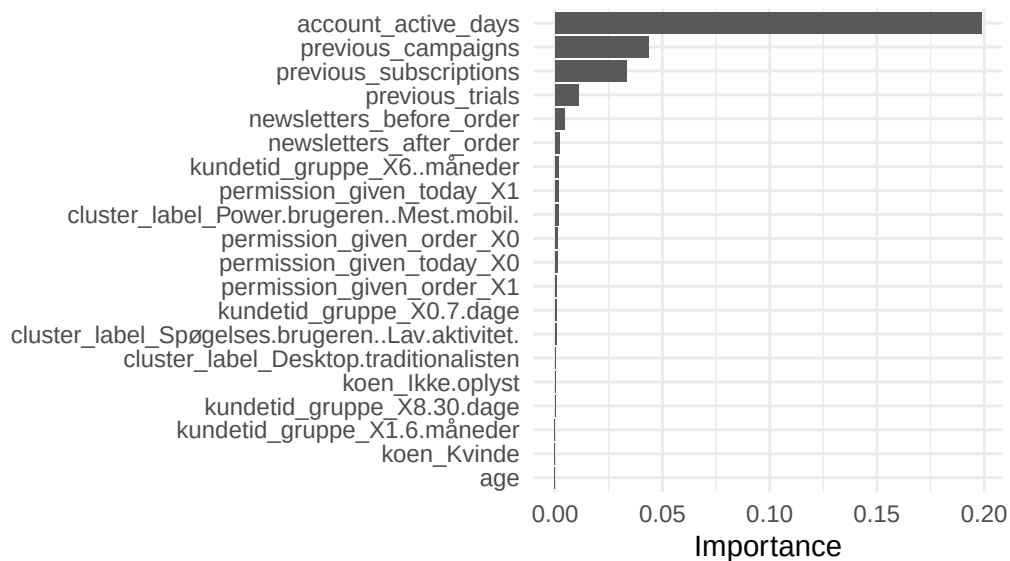
```

final_fit_early %>%
  extract_fit_parsnip() %>%
  vip(num_features = 20) +
  theme_minimal() +
  labs(title = "What Drives Early Churn?",
       subtitle = "Variables ranked by Importance (Decision Tree)")

```

What Drives Early Churn?

Variables ranked by Importance (Deci



```
# Best og worst case definition
# Extract fitted workflows
churn_model <- extract_workflow(final_fit)
early_model <- extract_workflow(final_fit_early)

scenarier <- tibble(
  case = c("Best case", "Average case", "Worst case"),

  # numeric
  account_active_days = c(2000, 450, 5),
  age = c(70, 45, 22),
  previous_subscriptions = c(1, 2, 0),
  previous_campaigns = c(1, 2, 0),
  previous_trials = c(0, 10, 0),
  newsletters_before_order = c(1, 2, 0),
  newsletters_after_order = c(1, 2, 0),

  # Kategoriske værdier
  koen = factor(
    c("Mand", "Mand", "Mand"),
    levels = levels(model_data_clean$koen)
  ),

  kundetid_gruppe = factor(
    c("6+ måneder", "6+ måneder", "0-7 dage"),
    levels = levels(model_data_clean$kundetid_gruppe)
  ),

  cluster_label = factor(
    c(
      "Veteranen (Høj historik)",
      "Desktop-traditionalisten",
      "Mobil-nykommeren"
```

```

    ),
    levels = levels(model_data_clean$cluster_label)
  ),

  permission_given_order = factor(
    c("1", "1", "0"),
    levels = levels(model_data_clean$permission_given_order)
  ),

  permission_given_today = factor(
    c("1", "0", "0"),
    levels = levels(model_data_clean$permission_given_today)
  ),
  churn = factor(c("0", "0", "0"), levels = c("1", "0")),
  early_churn = c(0, 0, 0)
)
# Forudseelser
# Main churn model
p_churn <- predict(
  churn_model,
  scenarier,
  type = "prob"
) %>%
  select(churn_prob = .pred_1)

# Early churn model
p_early <- predict(
  early_model,
  scenarier,
  type = "prob"
) %>%
  select(early_prob = .pred_1)

# Kombiner resultater med bind_cols
scenario_results <- bind_cols(
  scenarier,
  p_churn,
  p_early
) %>%
  mutate(
    total_risk =
      churn_prob +
      (1 - churn_prob) * early_prob
  ) %>%
  mutate(
    across(
      c(churn_prob, early_prob, total_risk),
      ~ round(.x * 100, 1)
    )
  ) %>%
  arrange(total_risk) %>%
  select(
    case,
    total_risk,
    churn_prob,

```

```

    early_prob,
    everything()
)

print(scenario_results, width = Inf)

# A tibble: 3 x 18
  case          total_risk churn_prob early_prob account_active_days  age
<chr>          <dbl>      <dbl>    <dbl>          <dbl> <dbl>
1 Best case      33.7        22.3      14.6            2000  70
2 Average case   66.5        49.7      33.4             450  45
3 Worst case    99.6         93       94.3              5  22
  previous_subscriptions previous_campaigns previous_trials
          <dbl>          <dbl>          <dbl>
1             1             1             0
2             2             2            10
3             0             0             0
  newsletters_before_order newsletters_after_order koen kundetid_gruppe
          <dbl>          <dbl> <fct> <fct>
1             1             1 Mand  6+ måneder
2             2             2 Mand  6+ måneder
3             0             0 Mand  0-7 dage
  cluster_label      permission_given_order permission_given_today churn
<fct>          <fct>          <fct>          <fct>
1 Veteranen (Høj historik) 1             1             0
2 Desktop-traditionalisten 1             0             0
3 Mobil-nykommeren         0             0             0
  early_churn
    <dbl>
1         0
2         0
3         0

```

6.3.1 Resultater og indsigt fra modellen

Vi har lavet to tidymodels modeller, en som forudser churn og en som forudser early churn. Vi har visualiseret de mest vigtige features ved hjælp af en Variable Importance-analyse.

Resultaterne viser, at antallet af dage kontoen har været aktiv, er den mest afgørende prædiktør for begge modeller. Derudover spiller historisk engagement, såsom antallet af tidligere kampagner, abonnementer og prøveperioder, en væsentlig rolle. Modtagelse af nyhedsbreve efter bestilling samt kundens alder har ligeledes en mærkbar indflydelse på sandsynligheden for fastholdelse.

6.3.2 Model 1 Churn

Accuracy (0,771): Modellens overordnede nøjagtighed er på 77,1 %. Det betyder, at modellen rammer rigtigt i mere end 3 ud af 4 tilfælde, når den skal klassificere, om en kunde forlader os eller bliver.

ROC-AUC (0,849): Dette er et af de vigtigste tal. En værdi på næsten 0,85 fortæller os, at modellen er rigtig god til at rangere kunderne efter risiko. Der er 85 % sandsynlighed for, at en tilfældig churmer får en højere risikoscore end en loyal kunde.

Sensitivitet / Recall (0,792): Modellen finder 79,2 % af alle de faktiske churnere. Det betyder, at vi fanger ca. 8 ud af 10 kunder, der er på vej væk. Dette er afgørende for Jyllands-Posten, da det giver os muligheden for at gribe ind over for langt de fleste potentielle opsigelser.

Specificitet (0,755): Modellen er også god til at identificere de loyale kunder (75,5 %). Det sikrer, at vi ikke sender for mange dyre fastholdelseskampanjer ud til kunder, som alligevel havde tænkt sig at blive.

F-measure (0,743): Dette tal er et vægtet gennemsnit, der viser balancen mellem præcision og sensitivitet. En værdi på 0,74 bekræfter, at modellen er velafbalanceret og ikke “overfitter” til den ene eller den anden side.

Vores confusion matrix for Model 1 viser en balanceret præcision med 84 korrekte churn-forudsigelser og 111 korrekte fastholdelser i testsættet. Tjek eventuelt også ROC kurven ovenfor, jo tættere den kommer på øverste venstre hjørne jo bedre, og vores ligner en god model men stadig ikke for urealistisk.

Fem forskellige klassifikationsmodeller blev testet – herunder Decision Tree, Logistisk Regression, SVM, XGBoost og Random Forest. Generelt fik vi de bedste resultater fra XGBoost og Random Forest, måske fordi de er bedre til at håndtere komplekse datamønstre.

6.3.3 Model 2: Early churn

Model 2 identificerer “early churn” blandt kunder, der fortsætter efter kampagnen, men falder fra inden for 90 dage.

Accuracy (0,837): Modellens overordnede nøjagtighed er på 83,7 %. Det betyder, at den rammer rigtigt i over 8 ud af 10 tilfælde, hvilket gør den til et meget pålideligt værktøj til at forudsige adfærd i den kritiske fase efter en kampagne.

ROC-AUC (0,869): Med en AUC-værdi på næsten 0,87 præsterer Model 2 endnu bedre end Model 1 til at adskille potentielle churnere fra loyale kunder. Der er næsten 87 % sandsynlighed for, at en kunde, der faktisk vil forlade os tidligt, får en højere risikoscore end en kunde, der bliver.

Sensitivitet / Recall (0,950): Dette er modellens stærkeste punkt. Den fanger hele 95 % af alle de faktiske “early churners”. Det betyder, at vi næsten med fuld sikkerhed kan identificere de kunder, der er i farezone for at smutte kort efter kampagnens udløb.

Specificitet (0,596): Modellens evne til præcist at identificere loyale kunder er på 59,6 %. Selvom den er lavere end sensitiviteten, prioriterer modellen her bevidst at “fange for mange” frem for at misse en kunde, der er ved at forlade os.

F-measure (0,888): Den høje F-værdi på 0,89 understreger, at modellen er særdeles effektiv til denne specifikke opgave, da den formår at balancere præcision og genfinding på et meget højt niveau.

Ligesom i model 1 sammenlignede vi de fem modeller, men Random Forest viste de bedste resultater. Det kunne tyde på der er mere komplekse mønstre i vores datasæt

6.3.4 Best, average og worst case scenarier

For at vise modellens værdi har vi opstillet tre scenarier, der illustrerer den samlede risiko for en kunde, beregnet som kombinationen af risikoen for umiddelbart churn og early churn.

Worst Case: En ung bruger (22 år) i segmentet “Mobil-nykommeren” med kun 5 dages aktivitet har en estimeret churn-risiko på hele 99,6 %. Her er behovet for øjeblikkelig intervention kritisk.

Average Case: En midaldrende mandlig bruger (45 år), der har været aktiv i over 6 måneder, men har en høj historik af prøveperioder, lander på en moderat risiko på 66,5 %.

Best Case: En ældre, loyal “Veteran” (70 år) med over 2.000 aktive dage og alle permissions givet har en lav risiko på 33,7 %.

6.4 Neural network churn model

Åbn <https://colab.research.google.com/>, lav “New Notebook” og importer JP_NeuralNetværk.ipynb. Derefter skal model_data_clean.rds indsættes i Filer under “sample_data”. Derefter kan koden køres.

Vi har her en rensningsblok og derefter fire modeller.

Model 01 fungerer som en basislinje og er implementeret som et simpelt neuralt netværk ved brug af Keras Sequential API. I koden ses dette bl.a. ved et to lag (`Dense(..., activation='relu')`) med L2-regularisering samt et outputlag med sigmoid-aktivering. Modellen kompiles med `adam` og `binary_crossentropy`. Adam er en effektiv optimeringsalgoritme der anvendes til at opdatere modellens vægte under træningen. Binary crossentropy er en loss funktion der er særligt egnet til binær klassifikation. Sigmoid aktivering omsætter outputtet til en sandsynlighedsværdi mellem 0 og 1.

Model 02 udvider baseline-modellen ved at inkludere et lag mere sammenlignet med model 01, samt `Batch-Normalization()`, og en stærkere l2 regularizer på 0,002 i stedet for 0,001. En højere L2 regularizer betyder at modellen i højere grad straffer store vægte under træningen. Dette øger modellens kapacitet til at lære mere komplekse mønstre i data og bidrager samtidig til en mere stabil træningsproces. Modellen repræsenterer derfor en mere avanceret arkitektur end Model 01.

Model 03 fokuserer på håndtering af ubalancerede data ved brug af SMOTE, hvilket i koden ses ved `SMOTE().fit_resample(X_train, y_train)`. Denne metode skaber et mere balanceret træningsdatasæt ved at generere syntetiske observationer af minoritetsklassen. Selve netværksarkitekturen minder om de tidligere modeller, men træningen foregår på et forbedret datagrundlag, hvilket styrker modellens evne til at identificere churn-kunder.

Model 04 anvender SMOTENC (`SMOTENC(categorical_features=...)`) som oversampling-teknik. Denne metode tager højde for både numeriske og kategoriske variabler og skaber dermed mere realistiske syntetiske observationer sammenlignet med standard SMOTE. Modellen kombinerer derfor en neural netværksarkitektur med en mere avanceret tilgang til håndtering af datasæt-ubalance.

Til sidst vurderes modellerne samlet ved hjælp af ROC-kurver, confusion matrices og centrale performance-mål såsom AUC, recall og accuracy.

Resultaterne viser, at Model 04 (SMOTENC) opnår den bedste samlede performance med den højeste AUC på 0,792 samt den højeste recall på 0,783. Det betyder, at modellen er bedst til at identificere churn-kunder, hvilket ofte er det vigtigste mål i churn-prediktion, da det er mere værdifuldt at fange flest mulige potentielle kunder, der risikerer at forlade virksomheden.

Model 01 (Basislinje) præsterer overraskende stærkt med en AUC på 0,779 og den højeste accuracy på 0,694, hvilket indikerer, at den simple model fortsat er konkurrencedygtig. Model 03 (SMOTE) opnår samme AUC på 0,779, men en højere recall på 0,755, hvilket viser, at SMOTE forbedrer modellens evne til at identificere churn-kunder på bekostning af en mindre reduktion i accuracy.

Model 02 (Complex) opnår de svageste resultater med en AUC på 0,759 og en recall på 0,613, hvilket tyder på, at øget modelkompleksitet alene ikke nødvendigvis fører til bedre performance. Tværtimod peger resultaterne på, at håndtering af datasæt-ubalance har større betydning end en mere kompleks netværksarkitektur.

Samlet set fremstår Model 04 (SMOTENC) som den stærkeste model, da den både leverer den bedste samlede diskriminationsevne (AUC) og den højeste recall. Resultaterne understreger dermed, at avancerede metoder til håndtering af ubalancerede data kan have en væsentlig positiv effekt på churn-prediktion.

7 Fejlkilder og fortolkning

En væsentlig metodisk udfordring i analysen er risikoen for data leakage (lækkage af fremtidig information), særligt i variabelen `account_active_days`, som er defineret som det samlede antal dage, en kunde har haft adgang gennem et aktivt JP-abonnement. Denne variabel er meget stærkt forklarende i modellen, men værdien er opgjort på tidspunktet for dataudtrækket og kan derfor indeholde information om kundens adfærd efter det tidspunkt, hvor churn-prediktionen i praksis skulle foretages. Modellen kan dermed indirekte få adgang til fremtidig information, hvilket kan føre til overvurdering af modellens præcision.

Vi har forsøgt, men ikke implementeret, at korrigere for dette ved at rekonstruere historiske værdier for `account_active_days`, `previous_trials`, `previous_subscriptions` og `previous_campaigns`, så disse i højere grad afspejlede den information, der reelt var tilgængelig på predictionstidspunktet. Dette viste sig imidlertid metodisk problematisk, da datasættet ikke indeholder tilstrækkelig historik til præcist at tilbagedatere disse variabler.

For at kunne rekonstruere historiske værdier korrekt, ville det blandt andet kræve mere detaljeret information om:

- start- og slutdatoer for hver abonnementsperiode
- eventuelle pauser eller midlertidige ophold i abonnementet
- fornyelsesdatoer og abonnementsændringer
- præcise datoer for prøveperioder og tidligere kampagner
- hvornår tidligere abonnementer blev oprettet og afsluttet

Da disse oplysninger ikke er tilgængelige i datagrundlaget, vil en tilbagedatering nødvendigvis bygge på antagelser, som kan introducere nye fejl og potentielt forvrænge de oprindelige variable. På den baggrund har vi vurderet, at en fuld historisk rekonstruktion ikke kan gennemføres på et fagligt forsvarligt grundlag med det nuværende datasæt.

Dette betyder, at modellen sandsynligvis overvurderer betydningen af enkelte variable, særligt `account_active_days`, og at modellens prædiktive performance derfor bør fortolkes med forsigtighed. Resultaterne giver stadig værdifuld indsigt i, hvilke kundedarakteristika der ser ud til at hænge sammen med churn, men modellens præcision vil forventeligt være lavere i en reel produktionssituation, hvor kun historisk tilgængelig information kan anvendes.

8 Konklusion

Projektet viser, at det er muligt at forudsige churn blandt kampagneabonnenter ved hjælp af Machine Learning med en relativt høj præcision, særligt ved brug af modeller som Random Forest og XGBoost.

En central begrænsning er risikoen for data leakage, især i variable som `account_active_days`, som kan indeholde information, der ikke reelt ville være tilgængelig på tidspunktet for en faktisk churn-prediktion. Dette betyder, at modellernes performance sandsynligvis er overvurderet, og at resultaterne skal tolkes med forsigtighed i en reel implementeringskontekst.

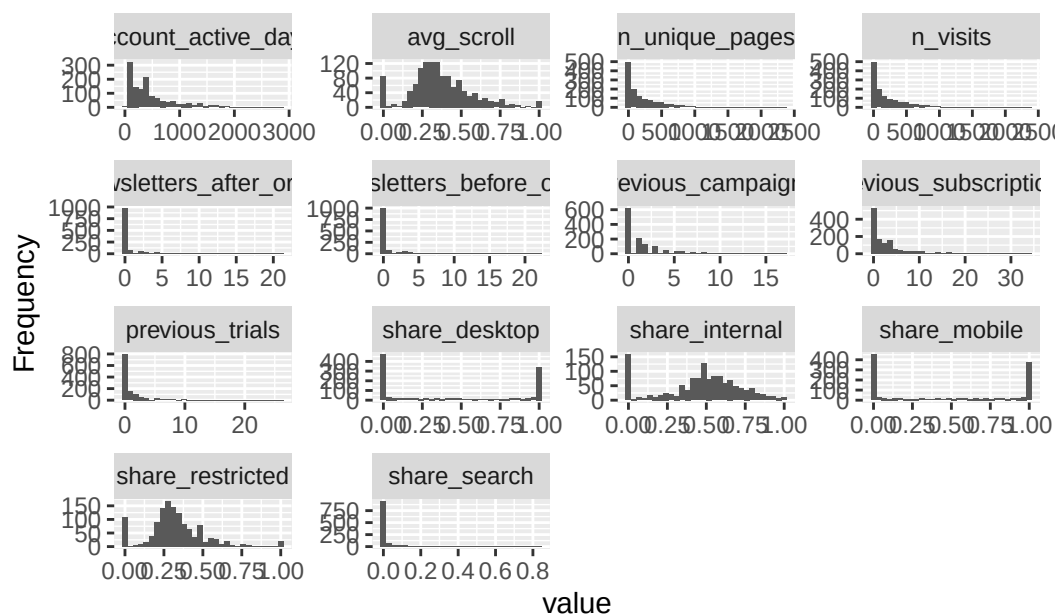
Udover det viser resultaterne, at især kundedadfærd og historik er centrale faktorer for churn. Modeller som Random Forest og XGBoost klarer sig som sagt bedst, og early churn-modellen er særligt stærk til at fange kunder i risikogruppen. Det neurale netværk er knap så godt i sammenligning, blandt andet fordi datamængden er relativt lille.

Kombinationen af ML-modeller, klyngeanalyse og et interaktivt dashboard giver Jyllands-Posten et brugbart værktøj til at identificere risikokunder, forstå kundemønstre og målrette fastholdelsesindsatser mere effektivt.

Samlet set kan metoden bidrage til lavere churn og mere datadrevne kampagne- og fastholdelsesstrategier.

9 Bilag 1 - Klyngeanalyse illustrationer

```
plot_histogram(cluster_input)
```



```
summary(pca)
```

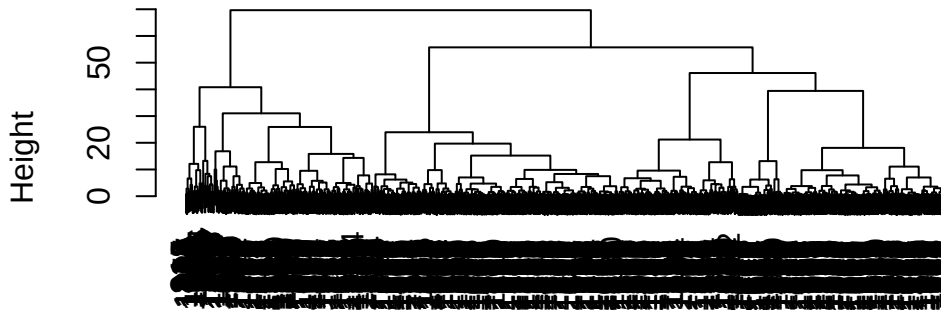
Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
Standard deviation	1.8808	1.5168	1.3714	1.2582	1.1846	1.01035	0.85395
Proportion of Variance	0.2527	0.1643	0.1343	0.1131	0.1002	0.07292	0.05209
Cumulative Proportion	0.2527	0.4170	0.5514	0.6644	0.7647	0.83758	0.88967

	PC8	PC9	PC10	PC11	PC12	PC13	PC14
Standard deviation	0.80168	0.64323	0.54330	0.34587	0.22191	0.15005	0.04036
Proportion of Variance	0.04591	0.02955	0.02108	0.00854	0.00352	0.00161	0.00012
Cumulative Proportion	0.93558	0.96513	0.98621	0.99476	0.99828	0.99988	1.00000

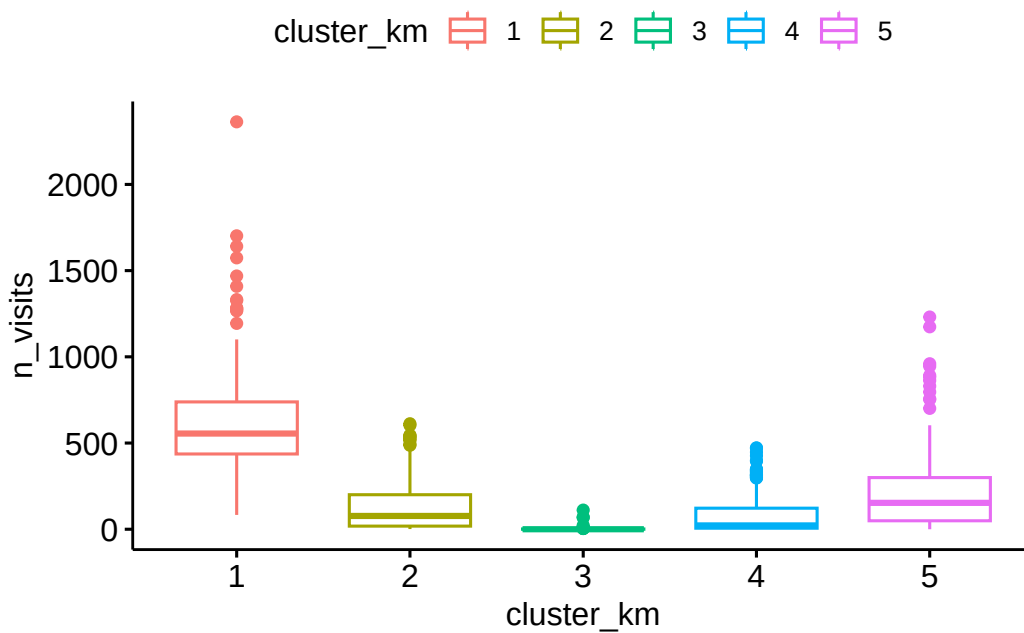
```
screplot(pca, type = "line")
abline(h = 1, col = "red", lty = 3)
```


Cluster Dendrogram

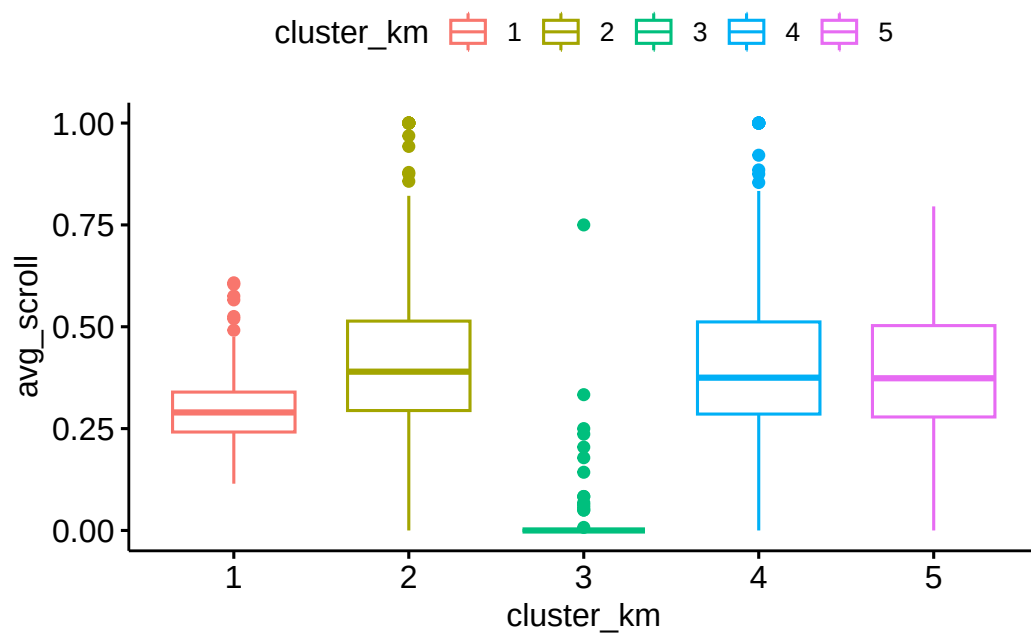


```
dist(pca$x[, 1:4])
hclust (*, "ward.D2")
```

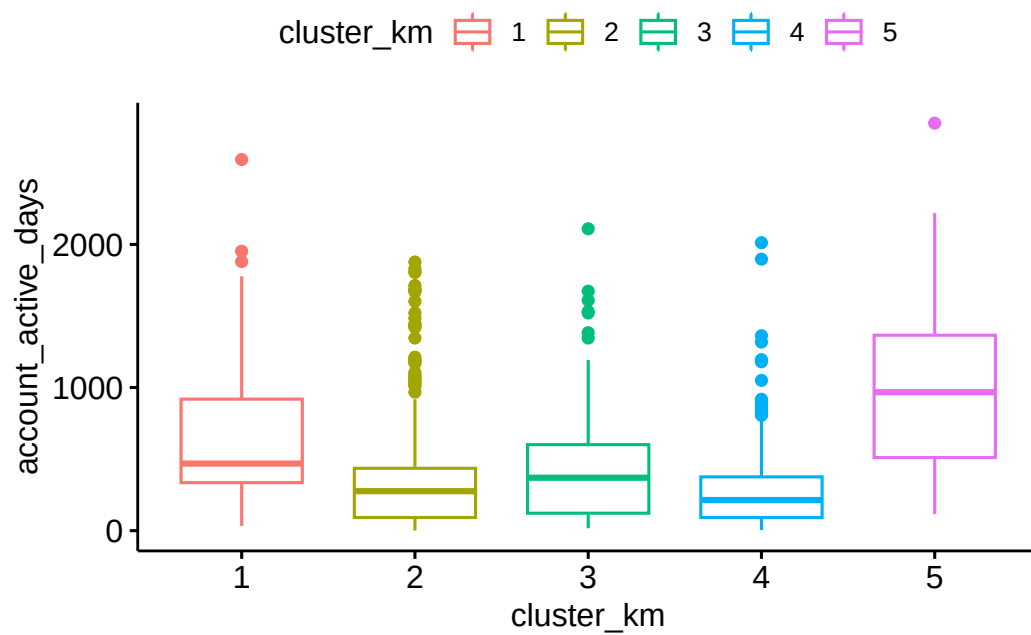
```
# Visualisiering
ggboxplot(customer_df, x = "cluster_km", y = "n_visits",
           color = "cluster_km")
```



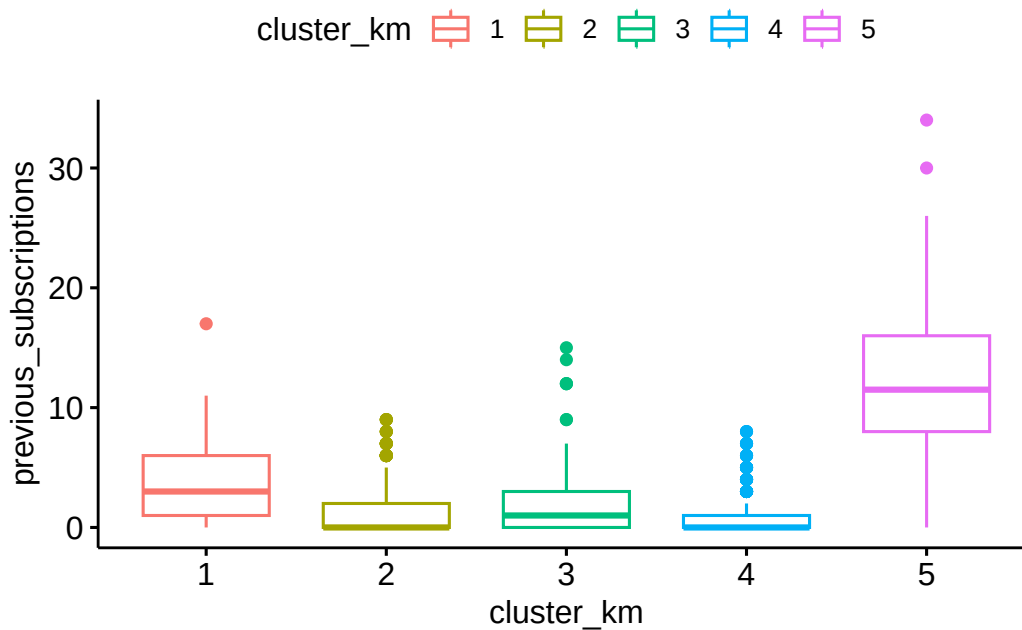
```
ggboxplot(customer_df, x = "cluster_km", y = "avg_scroll",
           color = "cluster_km")
```



```
ggboxplot(customer_df, x = "cluster_km", y = "account_active_days",
           color = "cluster_km")
```



```
ggboxplot(customer_df, x = "cluster_km", y = "previous_subscriptions",
           color = "cluster_km")
```



```
# ANOVA tests
```

```
summary(aov(n_visits ~ cluster_km, data = customer_df))
```

```

      Df   Sum Sq Mean Sq F value Pr(>F)
cluster_km    4 52749585 13187396    393 <2e-16 ***
Residuals  1254 42084141    33560
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(avg_scroll ~ cluster_km, data = customer_df))
```

```

      Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4   14.18   3.546  129.6 <2e-16 ***
Residuals  1254   34.31   0.027
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(account_active_days ~ cluster_km, data = customer_df))
```

```

      Df   Sum Sq Mean Sq F value Pr(>F)
cluster_km    4 63885784 15971446  107.5 <2e-16 ***
Residuals  1254 186284206  148552
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(previous_subscriptions ~ cluster_km, data = customer_df))
```

```

      Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4  15416   3854  387.5 <2e-16 ***
Residuals  1254  12473    10
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(previous_campaigns ~ cluster_km, data = customer_df))
```

```

              Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4   3324   831.0   251.1 <2e-16 ***
Residuals  1254   4151     3.3
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```
summary(aov(previous_trials ~ cluster_km, data = customer_df))
```

```

              Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4   4161   1040   258.7 <2e-16 ***
Residuals  1254   5042     4
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```
summary(aov(newsletters_before_order ~ cluster_km, data = customer_df))
```

```

              Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4   1551   387.8   119 <2e-16 ***
Residuals  1254   4088     3.3
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```
summary(aov(newsletters_after_order ~ cluster_km, data = customer_df))
```

```

              Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4   1607   401.9  109.7 <2e-16 ***
Residuals  1254   4595     3.7
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

10 Bilag 2 - Power BI Præsentation

For at få R script til at fungere, skal filstien ændres til din lokale computer. `readRDS("C:\Users\painb\Desktop\2ndsemester-JPcase\data\churn_model_rf.rds")`

11 Kildeliste

Chatgpt GPT-5.3, Claude Opus 4.7 og Google Gemini 3 er brugt som hjælpemiddel og idégenerering til kodning

Anthropic. 2026. "Claude AI Assistant". <https://claude.ai/>.

GDPR.dk. 2025. "GDPR omfatter alle virksomheder". <https://gdpr.dk/>.

Google. 2026. "Gemini AI Model". <https://gemini.google.com/>.

JP/Politikens Hus. 2025. "Privatlivspolitik". <https://jppol.dk/privatlivspolitik/>.

OpenAI. 2026. "ChatGPT". <https://chat.openai.com/>.